



**ROGUE LINUX: A DETERMINISTIC
BUILD SYSTEM AND COGNITIVE
PROCESS SUPERVISOR FOR MINIMAL
LINUX-BASED OPERATING SYSTEM
IMAGES**



A PROJECT REPORT

Submitted by

SRIDHARAN T (620822205302)

PRAVEEN RAJ G (620822205078)

SHANMUGA SRILAN T (620822205097)

MUGILAN K (620822205063)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

GNANAMANI COLLEGE OF TECHNOLOGY

NAMAKKAL – 637 018

ANNA UNIVERSITY: CHENNAI – 600 025

MAY 2025

BONAFIDE CERTIFICATE

Certified that this project report “**ROGUE LINUX: A DETERMINISTIC BUILD SYSTEM AND COGNITIVE PROCESS SUPERVISOR FOR MINIMAL LINUX-BASED OPERATING SYSTEM IMAGES**” is the bonafide work of **SRIDHARAN T (620822205302), PRAVEEN RAJ G(620822205078), SHANMUGA SRILAN T (620822205097) and MUGILAN K(620822205063)** who carried out the project work under my supervision.

SIGNATURE

Mrs.D.LATHA,

HEAD OF THE DEPARTMENT

Assistant professor,
Department of Information Technology
Gnanamani College of Technology
Namakkal – 637 018

SIGNATURE

Mrs.B.SARANYA,

SUPERVISOR,

Assistant professor
Department of Information Technology
Gnanamani College of Technology
Namakkal – 637 018

Submitted for the End Semester Project Work Viva-voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our deep sense of heartiest thanks to our beloved Chairman **Dr.T.ARANGANNAI**, beloved Chairman Smt. **P.MALALEENA ARANGANNAL**, and Vice Chairperson **Ms.MADUVANTHINIE ARANGANNAL**, Gnyanamani Educational Institutions, Namakkal, for given an opportunity to do and complete this project.

We would like to express our sincere thanks to Chief Administrative Officer **Dr.P.PREMKUMAR** for his support to bring the best in myself. We would like to convey our sincere thanks to our Principal **Dr.B.SANJAY GANDHI**, Gnanamani College of Technology for forwarding us to do our project and offering adequate duration to complete our project.

We extend thanks to our Executive Director **Dr.M.MADHESWARAN**, Gnanamani College of Technology, Namakkal, for his encouragement during our project work successfully.

We expand sincere thanks to our co-ordinator of school of computing, **Dr.S.GOPINATH**, Gnanamani College of Technology, Namakkal with deep sense of gratitude.

We are thankful to out Project Coordinator **Mrs.R.MEKALA**, Assistant Professor, for providing good suggestions and help me to make this project a success.

We extend earnest & sincere thanks to my Project Guide **Mrs.B.SARANYA**, Assistant Professor for her kind guidance and encouragement during this project.

We would also like to express my thanks to all staff members of our department, friends who helped me directly and indirectly in all aspects of the project work to get completed successfully.

[SRIDHARAN T]

[PRAVEEN RAJ G]

[SHANMUGA SRILAN
T]

[MUGILAN K]



Gnanamani College of Technology

(AUTONOMOUS)

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai
NH-7, A.K. Samuthiram, Pachal – 637018, Namakkal (Dt.), Tamilnadu, India.
Web : www.gct.org.in, E-Mail : info@gct.org.in, Contact No. 7598293888, 7598293999

Institute Vision:

To emerge as a globally recognized technical institution to produce ethical engineers,
researchers, administrators and entrepreneurs

Institute Mission:

- To provide state-of-the-art infrastructure to create an effective learning environment for students
 - To collaborate with leading industries and academia to empower students to meet global standards
 - To foster an enterprising environment that encourages innovation and entrepreneurial activities among students
-

Department Vision: (IT)

To be the department that imparts professional computing training and makes competent engineer to work in the emerging areas of information technology field.

Mission Statements: (IT)

- To prepare competent engineers and adapt to the dynamic needs of industries.
 - To pave way to the enrichment of knowledge and skills using latest technologies in the diverse domain of information technology.
 - Inculcate strong ethical values and professionalism to serve society while concurrently updating knowledge and skills through life-long learning.
-



Gnanamani College of Technology

(AUTONOMOUS)

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai
NH-7, A.K. Samuthiram, Pachal – 637018, Namakkal (Dt.), Tamilnadu, India.
Web : www.gct.org.in, E-Mail : info@gct.org.in, Contact No. 7598293888, 7598293999

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

Graduates of Information Technology will

- **PEO-1:** Be proficient in utilizing the fundamental knowledge of various streams in engineering and technology.
- **PEO-2:** Think logically and to pursue lifelong learning to understand technical issues related to computing systems and to provide optimal solutions.
- **PEO-3:** Design and develop hardware and software system by understanding the importance of social, business and environmental needs in the social contexts.

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM OUTCOMES (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

Graduates of the program will be able to

- **PSO-1:** Apply the mathematical and the computing knowledge to identify and provide solutions for computing problems.
- **PSO-2:** Design and develop computer programs in the areas related to algorithms, networking, web design and data analytics of varying complexity.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	V
	ABSTRACT	Viii
	PROBLEM STATEMENT	xi
	LIST OF FIGURES	1
	LIST OF ABBREVIATIONS	1
1.	INTRODUCTION	2
	1.1.Overview	3
	1.2.Objectives	4
	1.3.Scope of the Project	4
2.	LITERATURE SURVEY	5
3.	SYSTEM ANALYSIS	6
	3.1.Existing System	6
	3.1.1. Disadvantages of Existing System	8
	3.2.Proposed System	15
	3.2.2. Advantages of Proposed System	16
4.	SYSTEM SPECIFICATION	16
	Hardware Requirements	16
	Software Requirements	16
5.	SOFTWARE DESCRIPTION	17
	C Programming Language (C11)	17
	Rust Programming Language	17
	TOML Configuration Format	17
6.	SYSTEM DESIGN	18
	System Architecture	18
	Data Flow Diagram	19
	DFD Level 0	20
	DFD Level 1	20

	UML Diagrams	21
	Use Case Diagram	21
	Class Diagram	22
	Sequence Diagram	23
	Component Diagram	23
7.	PROJECT DESCRIPTION	24
	Module Description	24
	cogman-planner Module	29
	cogman-executor Module	30
	cogman-supervisor Module	
	cogman-ctl Module	
	Messenger IPC Module	
	Rootfs Bootstrap Module	
8.	SYSTEM TESTING	
	Software Testing	
	Unit Testing	
	Integration Testing	
	System Testing	
	Functional Testing	
	Non-Functional Testing	
	User Acceptance Testing	
	Test Cases	
9.	CONCLUSION AND FUTURE ENHANCEMENT	
	Conclusion	
	Future Enhancement	
10.	APPENDIX	
	Source Code	
	REFERENCES	

TABLE OF CONTENTS

	TITLE	PAGE NO.
	ABSTRACT	viii
	PROBLEM STATEMENT	ix
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xi
1	INTRODUCTION	1
1.1	Overview	1
1.2	Objectives	3
1.3	Scope of the Project	4
2	LITERATURE SURVEY	5
3	SYSTEM ANALYSIS	10
3.1	Existing System	10
3.1.1	Disadvantages of Existing System	11
3.2	Proposed System	12
3.2.1	Advantages of Proposed System	13
4	SYSTEM SPECIFICATION	14
4.1	Hardware Requirements	14
4.2	Software Requirements	15
5	SOFTWARE DESCRIPTION	16
5.1	C Programming Language (C11)	16
5.2	Rust Programming Language	17
5.3	TOML Configuration Format	18
6	SYSTEM DESIGN	19
6.1	System Architecture	19
6.2	Data Flow Diagram	22
6.2.1	DFD Level 0	22
6.2.2	DFD Level 1	23
6.3	UML Diagrams	24
6.3.1	Use Case Diagram	24
6.3.2	Class Diagram	26
6.3.3	Sequence Diagram	28
6.3.4	Component Diagram	30
7	PROJECT DESCRIPTION	32
7.1	Module Description	32
7.1.1	cogman-planner Module	32
7.1.2	cogman-executor Module	35
7.1.3	cogman-supervisor Module	38
7.1.4	cogman-ctl Module	41
7.1.5	Messenger IPC Module	43
7.1.6	Rootfs Bootstrap Module	44
8	SYSTEM TESTING	46
8.1	Software Testing	46
8.1.1	Unit Testing	46
8.1.2	Integration Testing	47
8.1.3	System Testing	48
8.1.4	Functional Testing	49

8.1.5	Non-Functional Testing	50
8.1.6	User Acceptance Testing	51
8.2	Test Cases	52
9	CONCLUSION AND FUTURE ENHANCEMENT	55
9.1	Conclusion	55
9.2	Future Enhancement	56
10	APPENDIX	57
10.1	Source Code	57
	REFERENCES	63

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Rogue Linux Build Pipeline Overview	2
1.2	Cogman Runtime Architecture	3
6.1	Overall System Architecture	20
6.2	CGM2PLAN Binary Plan Format	21
6.3	DFD Level 0 — Context Diagram	22
6.4	DFD Level 1 — Build Subsystem	23
6.5	Use Case Diagram — Build Subsystem	25
6.6	Use Case Diagram — Runtime Supervisor	25
6.7	Class Diagram — Planner	27
6.8	Class Diagram — Supervisor	27
6.9	Sequence Diagram — Package Build Flow	29
6.10	Sequence Diagram — Service Start Flow	29
6.11	Component Diagram	31
7.1	cogman-planner Dependency Graph Resolution	33
7.2	Dependency Topological Sort Example	34
7.3	cogman-executor Step Execution Loop	36
7.4	Path Traversal Guard Logic	37
7.5	cogman-supervisor Signal Handling (Self-Pipe)	39
7.6	Service Lifecycle State Machine	40
7.7	cogman-ctl Unix Socket Protocol	42
7.8	Messenger IPC Protocol (TLV Format)	44
7.9	Rootfs Directory Layout	45
8.1	Service Boot Sequence on Minimal Rootfs	48

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
API	Application Programming Interface
BFD	Binary File Descriptor
CGM	Cogman
CLI	Command Line Interface
DAG	Directed Acyclic Graph
DFD	Data Flow Diagram
ELF	Executable and Linkable Format
ER	Entity Relationship
FD	File Descriptor
GCC	GNU Compiler Collection
IPC	Inter-Process Communication
LLM	Large Language Model
mmap	Memory-Mapped File
OS	Operating System
PID	Process Identifier
POSIX	Portable Operating System Interface
rootfs	Root Filesystem
SIGCHLD	Signal Child
SIGTERM	Signal Terminate
TOML	Tom's Obvious Minimal Language
UML	Unified Modelling Language
UAT	User Acceptance Testing
QEMU	Quick Emulator
SHA	Secure Hash Algorithm
TLV	Type-Length-Value
UDS	Unix Domain Socket

ABSTRACT

Rogue Linux is a deterministic, metadata-driven infrastructure for constructing minimal Linux-based operating system images. The core innovation is Cogman (Cognitive Manager), a unified toolchain that spans both the build phase and the runtime phase of a Linux system. During the build phase, `cogman-planner` — implemented in the Rust programming language — reads declarative TOML package definitions, resolves a directed acyclic dependency graph using topological sort, enforces filesystem and network security policies, and emits a compact binary execution plan in the CGM2PLAN format. The complementary `cogman-executor`, implemented in C11, memory-maps the plan file and executes each typed step operation (`OP_EXEC`, `OP_MKDIR`, `OP_COPY`, `OP_VERIFY`, `OP_CLEANUP`) with path traversal protection and optional SHA-256 verification.

During the runtime phase, `cogman-supervisor` acts as PID 1 — the first process executed by the Linux kernel — and manages the complete lifecycle of system services. It parses service definition files, implements an asynchronous `SIGCHLD` self-pipe pattern for safe child-process reaping, enforces dependency ordering, and supports three restart policies (never, on-failure, always). A Unix domain socket control interface (`cogman-ctl`) allows operators to list, start, stop, and restart services at runtime. The messenger subsystem provides typed IPC (heartbeat, alert, log) between cogman components using a fixed 16-byte header protocol.

The system achieves a $56\times$ improvement in plan resolution time (8 ms vs 450 ms), $21\times$ reduction in peak memory (4 MB vs 85 MB), and a $50\times$ reduction in execution overhead per step compared to the reference Python-based legacy implementation. A minimal bootable rootfs was constructed weighing approximately 6.3 MB that boots under QEMU and successfully completes a four-stage verification sequence exercising all code paths. This

project demonstrates that a deterministic, safe, and high-performance system software stack can be built using modern Rust and C11 without external dependencies beyond the Linux kernel and the C standard library.

PROBLEM STATEMENT

Modern embedded Linux systems, container base images, and minimal operating environments demand a reproducible, auditable method for constructing a root filesystem from source packages. Existing solutions such as Buildroot and Yocto Project address this need at the cost of enormous complexity: Buildroot requires a dedicated Python and Bash toolchain exceeding 200 MB; Yocto requires days of build time and specialized expertise. Neither solution provides a lightweight, self-hosting init daemon that doubles as a package manager.

A secondary problem is that existing init systems (SysVinit, OpenRC, systemd) are not designed for minimal embedded environments. systemd requires udev, D-Bus, and tens of shared libraries; SysVinit relies on fragile shell scripts. Neither provides a programmable control interface, dependency-aware service ordering, or integration with a build pipeline.

The absence of a unified toolchain that spans the build phase and the runtime phase forces developers to combine disparate tools (Makefile, Docker, shell scripts, busybox init, systemd) with no common metadata format, security policy model, or error-diagnosis capability. This fragmentation makes reproducibility difficult and security auditing nearly impossible.

Rogue Linux addresses these problems by providing a single coherent system: a Rust-based planner that consumes TOML package definitions and emits typed binary plans; a C-based executor that runs those plans with path-safety enforcement; and a C-based PID-1 supervisor with a text-protocol control socket — all operating on a rootfs that can be described in a single manifest file.

CHAPTER 1

INTRODUCTION

1.1 Overview

Rogue Linux is not a Linux distribution in the conventional sense. It is an infrastructure for building one. Given a set of declarative package definitions written in TOML, Rogue Linux produces a reproducible root filesystem capable of booting under QEMU or on bare metal, with Cogman serving as the init process. The name Cogman is an abbreviation for Cognitive Manager, reflecting the system's design goal of being a self-aware process supervisor that understands the relationships between the services it manages.

The system is divided into two conceptually distinct halves. The build half operates on the developer's host machine and converts package metadata into a staged root filesystem. The runtime half operates on the target system after the kernel has booted and takes over all process management duties from that point forward. This clean separation ensures that no build-time dependencies leak into the runtime environment and that the final rootfs contains only what was explicitly requested.

Figure 1.1 illustrates the build pipeline. A package author writes a TOML file describing the package's name, version, source archive, build steps, installer steps, and security policy. The cogman-planner binary reads this file, loads any declared dependencies recursively, performs a topological sort of the dependency graph, validates all fields against the schema, applies policy enforcement, and writes a binary plan file. The cogman-executor binary reads the plan, maps it into memory, and executes each step in order.

[Figure 1.1: Rogue Linux Build Pipeline Overview]

Figure 1.1: Rogue Linux Build Pipeline Overview

Figure 1.2 illustrates the runtime architecture. After the kernel finishes its initialization sequence, it executes `/sbin/init`, which is a symbolic link to `cogman-supervisor`. The supervisor reads all `*.service` files from `/etc/cogman/services/`, determines which services have no dependencies, and starts them immediately. Services with dependencies are started as soon as their declared dependencies reach a Running or Done state. A 100-millisecond polling loop checks for dead children (via a `SIGCHLD` self-pipe), accepts control commands from `cogman-ctl`, and respawns services that have elapsed their restart delay.

[Figure 1.2: Cogman Runtime Architecture]

Figure 1.2: Cogman Runtime Architecture

1.2 Objectives

1. To design and implement a dependency-aware binary plan generator (`cogman-planner`) in Rust that consumes TOML package metadata and produces typed, auditable binary execution plans.
2. To implement a minimal, zero-overhead plan executor (`cogman-executor`) in C11 that executes plans via memory-mapped I/O and enforces path traversal security at every copy operation.
3. To build a PID-1 capable process supervisor (`cogman-supervisor`) in C11 that manages service lifecycle with dependency ordering, restart policies, and a Unix domain socket control interface.
4. To design a typed IPC protocol (messenger) that allows cogman components to exchange structured messages without relying on shared memory or external middleware.
5. To construct a minimal bootable rootfs weighing under 10 MB that demonstrates all code paths in a four-stage verification sequence.

6. To achieve measurable performance improvements over the Python-based reference implementation in plan resolution time, peak memory usage, and per-step execution overhead.

1.3 Scope of the Project

The scope of the project encompasses the full toolchain from package metadata to a booted minimal Linux system. The build-time scope includes: TOML metadata schema definition, Rust-based dependency graph resolution and topological sort, binary plan format design (CGM2PLAN), policy enforcement (filesystem write-path restriction and network outbound control), plan caching using content-addressed FNV-1a hashing, and C11-based plan execution with SHA-256 verification.

The runtime scope includes: PID-1 supervisor with self-pipe SIGCHLD handling, INI-based service file parsing, three service types (process, oneshot, forking), three restart policies (never, on-failure, always), dependency-gated service start, Unix domain socket control protocol, and messenger-based IPC for heartbeat and alert messages.

The project does not include a graphical user interface, network stack configuration, package signing or verification infrastructure, persistent package database, or support for architectures other than x86_64 in the current release. AI-assisted failure diagnosis via a locally-running LLM is implemented as an optional feature gated by a compile-time feature flag and is not part of the core system.

CHAPTER 2

LITERATURE SURVEY

2.1 Poettering, L

Poettering, L. and Sievers, K. (2010). systemd - Rethinking PID 1. Proceedings of the Linux Symposium, Ottawa, Canada.

This foundational paper introduced the concept of socket-based activation and parallel service startup in systemd. The authors argued that traditional SysV init scripts serialize startup unnecessarily and that dependency tracking should be declarative rather than implicit in script order. Cogman-supervisor adopts the declarative dependency model from this work but discards the D-Bus and udev dependencies, making it suitable for environments where systemd's 1.5 MB binary and runtime library requirements are unacceptable. The self-pipe SIGCHLD pattern used in cogman-supervisor was inspired by the async-signal-safe design principles discussed in this paper.

2.2 Hennessy, J

Hennessy, J. L. and Patterson, D. A. (2019). Computer Organization and Design RISC-V Edition: The Hardware Software Interface. Morgan Kaufmann Publishers.

This textbook provided the theoretical foundation for understanding binary file formats and memory-mapped I/O. The CGM2PLAN binary plan format uses fixed-size header and record structures directly analogous to ELF section headers described in this reference. The use of `mmap()` in cogman-executor for zero-copy plan reading was informed by the memory hierarchy and I/O subsystem chapters. The alignment rules for 64-bit fields in the plan header follow the ABI conventions discussed in the architecture sections.

2.3 Klabnik, S

Klabnik, S. and Nichols, C. (2022). The Rust Programming Language, 2nd Edition. No Starch Press, San Francisco.

The Rust programming language's ownership, borrowing, and lifetime system provides compile-time guarantees against buffer overflows, use-after-free, and data races. Cogman-planner is implemented in Rust specifically to leverage these guarantees in the security-critical dependency resolution and policy enforcement paths. The `serde` and `toml` crates used for metadata deserialization are discussed in the ecosystem chapter. The error propagation pattern using the `?` operator, used extensively in `cogman-planner`, is derived from idiomatic Rust as presented in this reference.

2.4 Kerrisk, M

Kerrisk, M. (2010). The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.

This comprehensive reference was the primary source for POSIX signal handling, fork/exec semantics, and Unix domain socket programming used throughout the `cogman-supervisor` and `cogman-ctl` implementations. The self-pipe trick for `SIGCHLD` handling (Chapter 63), `waitpid()` semantics for zombie prevention (Chapter 26), and `SO_RCVTIMEO` socket option for read timeout (Chapter 56) are directly referenced in the implementation. The discussion of PID 1 responsibilities — including orphan process adoption and zombie reaping — was essential for `cogman-supervisor`'s main loop design.

2.5 Torvalds, L

Torvalds, L. et al. (2024). Linux Kernel Documentation: Init and Early Userspace. kernel.org documentation project.

The kernel documentation specifies the precise contract between the kernel and PID 1: the `init` process must not exit (kernel panic), must reap all orphaned processes, and receives `SIGTERM` on system halt. `Cogman-supervisor`'s shutdown path (stopping all services, closing the control socket, returning from `main`) was designed in accordance with

this specification. The `console=ttyS0` kernel command line parameter used in the QEMU boot configuration is documented [here](#).

2.6 Peter, J

Peter, J. B., Silberschatz, A., and Galvin, P. B. (2018). Operating System Concepts, 10th Edition. John Wiley and Sons.

The process scheduling and inter-process communication chapters provided the theoretical grounding for cogman's service model. The coverage of signals as a lightweight notification mechanism, pipes as communication channels, and Unix domain sockets as high-bandwidth local IPC informed the design of the messenger subsystem and the control socket protocol. The treatment of zombie processes and the relationship between parent and child process accounting was particularly relevant to the supervisor's SIGCHLD handling strategy.

2.7 Lumina, B

Lumina, B. et al. (2022). Buildroot: The Embedded Linux Build System. Buildroot Project Documentation, buildroot.org.

Buildroot is the closest existing system to Rogue Linux in terms of goals. The study of its Kconfig-based package system revealed significant complexity arising from its need to support over 2,000 packages across multiple architectures. Rogue Linux's TOML-based package format deliberately limits itself to the metadata needed for plan emission, trading generality for simplicity and type-safety. Buildroot's cross-compilation toolchain approach was studied to understand the staging root (PKGROOT) concept adopted in cogman-executor.

2.8 Coady, C

Coady, C. (2019). musl libc: A New Standard C Library. musl-libc.org technical overview.

musl libc is the C standard library used in the Rogue Linux minimal rootfs. Its design goals — correctness, safety, and small static linking size — align with Rogue Linux's

objectives. The study of musl's symbol table and static linking behavior informed the decision to statically link cogman-supervisor and cogman-executor in the rootfs, eliminating the dynamic linker as a potential point of failure during early boot.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

The existing approach to building minimal Linux root filesystems relies on a combination of general-purpose tools without a unified metadata model or execution engine. The dominant tools in this space are Buildroot, Yocto Project / OpenEmbedded, and hand-written shell scripts. Each carries significant limitations when applied to the use case of constructing a minimal, reproducible OS image for embedded or container environments.

Buildroot is a Kconfig-based build system that generates root filesystems from source. It requires a host system with make, gcc, Python 3, and several development libraries. A typical Buildroot build for a minimal x86_64 rootfs takes 20–60 minutes and produces a build tree exceeding 2 GB. Package metadata is stored in per-package Makefiles using a proprietary macro language. There is no typed validation of package definitions at planning time; errors surface as build failures deep in the make dependency graph.

Yocto Project / OpenEmbedded uses BitBake, a Python-based build engine, with package metadata in .bb (BitBake recipe) files. While Yocto is extremely powerful and supports thousands of packages, its complexity is a substantial barrier: a fresh Yocto setup requires downloading several gigabytes of metadata layers, and its learning curve is measured in weeks. Incremental builds depend on Yocto's internal hash-based caching system, which is difficult to understand and debug.

Alpine Linux uses APK (Alpine Package Keeper), a binary package manager with a simple format. While APK is efficient, it requires a running Alpine Linux system to build

packages and has no first-class concept of a build plan that can be audited or cached independently of the package build itself.

For process supervision at runtime, the existing options are SysVinit (shell-script-based, no dependency tracking), runit (minimal but no structured metadata), OpenRC (Bash-based with RC scripts), Busybox init (very minimal, no control socket), and systemd (feature-complete but requires D-Bus, udev, and 1.5+ MB of dependencies).

3.1.1 Disadvantages of Existing System

- No unified metadata format spanning build-time and runtime concerns.
- Build systems require hundreds of megabytes of host dependencies.
- Package definitions are in proprietary Makefile/BitBake formats with no schema validation.
- No typed binary plan format — errors manifest at execution time rather than planning time.
- No built-in path traversal protection in copy operations.
- Init systems either require significant dependencies (systemd) or lack dependency ordering (runit, busybox init).
- No programmatic control interface for the init daemon during boot.
- Performance: Python-based plan resolution takes 400–500 ms; memory usage during build exceeds 80 MB.
- No AI-assisted failure diagnosis when a build or service start fails.
- Difficult to audit: shell scripts execute arbitrary commands with no type system.

3.2 Proposed System

The proposed system, Rogue Linux / Cogman, addresses all of the above disadvantages through a two-component architecture sharing a common metadata model. The system is designed around three principles: determinism (the same inputs always produce the same roots), safety (typed operations and path validation prevent accidental damage), and minimalism (no dependency not strictly necessary for correct operation is included).

The build half is implemented as cogman-planner, a Rust binary that reads TOML package definitions, validates them against a strongly-typed schema using the `serde` crate, resolves dependencies using a directed acyclic graph with topological sort, and emits a

CGM2PLAN binary plan. The binary plan format uses fixed-size 64-byte headers and 128-byte step records aligned on 8-byte boundaries, making it directly memory-mappable by the executor with zero deserialization cost.

The execution half is cogman-executor, a C11 binary that maps the plan into memory and executes each step operation through a typed dispatch table. Path traversal is checked on every `OP_COPY` operation by scanning each path component for bare `'..'` entries. The executor reports step progress using a structured logging system and exits with a non-zero code on the first step failure, making it composable with shell scripts and CI pipelines.

The runtime half is cogman-supervisor, a C11 PID-1 daemon that reads INI-format service files, builds a dependency graph, and manages service processes through a 100ms main loop. The self-pipe `SIGCHLD` pattern ensures that signal handling is async-signal-safe and composable with the main loop's `select()`-based I/O polling. The cogman-ctl client communicates with the supervisor over a Unix domain socket using a simple newline-terminated text protocol.

3.2.1 Advantages of Proposed System

- Unified TOML metadata format covers identity, dependencies, build steps, installer steps, and security policy.
- Rust's type system validates package definitions at schema load time — errors surface before any build step executes.
- CGM2PLAN binary format is zero-allocation, zero-parsing — the executor reads it via `mmap` with no heap allocation.
- Path traversal guard on every `OP_COPY` operation prevents accidental writes outside the staging root.
- PID-1 supervisor with dependency-aware service ordering eliminates race conditions in service startup.
- Self-pipe `SIGCHLD` pattern ensures async-signal-safe child reaping compatible with `select()`-based main loop.
- Unix domain socket control interface allows runtime service management without shell scripting.
- Content-addressed plan caching (FNV-1a hash) skips re-planning when package metadata is unchanged.

- 56× faster plan resolution, 21× lower memory usage, 50× lower per-step overhead vs. Python reference.
- Optional AI advisor (Qwen2.5-3B via Ollama) provides structured failure diagnosis.
- Minimal rootfs (6.3 MB) boots under QEMU in under 2 seconds with all four verification services passing.

CHAPTER 4

SYSTEM SPECIFICATION

4.1 Hardware Requirements

The following hardware configuration is required to build, test, and run the Rogue Linux / Cogman system:

Component	Minimum Requirement	Recommended
Processor	Intel/AMD x86_64, 2 cores	Intel/AMD x86_64, 4+ cores
RAM	4 GB	8 GB or more
Storage	20 GB free disk space	50 GB SSD
Architecture	x86_64 (64-bit only)	x86_64
Virtualization	QEMU 6.0+ (for testing)	QEMU 8.0+ with KVM
Network	Not required for build	Optional (for source fetch)

4.2 Software Requirements

The following software is required on the host build machine:

Software	Version	Purpose
Linux OS (host)	Kernel 5.15+	Build environment
GCC	11.0 or later	Compile cogman-executor, cogman-supervisor
Rust (rustup)	1.78 or later	Build cogman-planner
GNU Make	4.3+	Top-level build orchestration
QEMU	6.2+	Run minimal rootfs for testing
BusyBox (static)	1.36.1	Shell and coreutils in rootfs
musl-libc (optional)	1.2.5	Alternative C library for static linking
Python 3	3.10+	Test runner and finalize_rootfs.py
Ollama (optional)	0.1.30+	AI advisor backend
git	2.40+	Version control

The build system has been verified on Kali Linux 6.x (x86_64), Debian 12 (Bookworm), and Ubuntu 22.04 LTS. All C sources are compiled with `-Wall -Wextra -O2 -std=c11 -D_POSIX_C_SOURCE=200809L`. Rust crates are compiled with `cargo build --release`, producing position-independent executables with link-time optimization enabled.

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 C Programming Language (C11)

The C11 standard (ISO/IEC 9899:2011) is used for cogman-executor and cogman-supervisor. C was chosen for these components because they interact directly with Linux kernel interfaces (fork, exec, waitpid, mmap, socket, bind, listen, accept) and must execute in the constrained environment of an early-boot rootfs where no Rust runtime or garbage collector is available. C11 provides `_Atomic` types and the `<stdint.h>` fixed-width integer types used in the plan header structs, ensuring correct layout across compilers.

The POSIX 2008 feature set (`_POSIX_C_SOURCE 200809L`) is enabled at compile time, providing access to `pselect()`, `ppoll()`, `sigaction()`, and other async-signal-safe interfaces required for the supervisor's signal handling strategy. All C code is compiled with `-Wall -Wextra -Wpedantic` to catch potential issues at compile time.

Key C11 features used: designated initializers in struct initialization, `_Static_assert` for compile-time struct size verification, `snprintf` for all string formatting (preventing buffer overflows), and the `restrict` keyword on pointer parameters in the plan I/O functions.

5.2 Rust Programming Language

Rust 1.78+ is used for cogman-planner. Rust's ownership model provides compile-time guarantees of memory safety (no buffer overflows, no use-after-free, no double-free) and thread safety (no data races). These guarantees are critical in the planner, which handles untrusted TOML input from package authors and resolves potentially circular dependency graphs.

The key crates used are: clap 4.x (command-line argument parsing), serde + toml (TOML deserialization into typed Rust structs), and a custom graph module for DAG construction and topological sort. The binary plan emitter writes directly to a `Vec<u8>` using fixed-size struct byte representations, ensuring the output matches the C-side reader's expectations without any external serialization library.

Rust's `Result<T, E>` type is used throughout the planner pipeline for explicit error propagation. The `?` operator propagates errors up the call stack to the top-level handler, which formats a human-readable message via the butler module's styled output functions. This ensures that no error is silently swallowed and that the user always receives a diagnostic message when planning fails.

5.3 TOML Configuration Format

Tom's Obvious Minimal Language (TOML) v1.0 is used as the package metadata format. TOML was chosen over JSON and YAML because it has unambiguous data types, is human-readable without mandatory indentation, and has a stable 1.0 specification. The `serde_toml` crate in Rust maps TOML documents directly to strongly-typed Rust structs, and any missing required field or type mismatch is reported as a structured error at load time.

The package metadata schema defines five top-level sections: `[identity]` (name, version, category, source), `[build]` (system type and shell commands), `[installer]` (installation steps and optional file verification), and `[policy]` (filesystem read/write restrictions and network outbound control). Service files use an INI-compatible subset of TOML with `[service]` and `[env]` sections.

CHAPTER 6

SYSTEM DESIGN

6.1 System Architecture

The Rogue Linux / Cogman architecture is organized into two distinct phases, each containing a planner component and an executor component. This separation enforces a clean boundary between the concerns of plan validation (done in a memory-safe language at planning time) and plan execution (done in a minimal C runtime at execution time).

The build-phase architecture begins with the package author writing a TOML metadata file. The cogman-planner binary reads this file along with all transitive dependency files, validates the complete dependency graph, and emits a CGM2PLAN binary. The CGM2PLAN file contains a 64-byte header, N step records of 128 bytes each, and a variable-length string table. The cogman-executor maps this file into memory using `mmap()` and executes each step record sequentially.

The runtime-phase architecture places cogman-supervisor at PID 1. On startup it opens `/etc/cogman/services/`, parses all `*.service` files, and builds an in-memory service table. It then enters a 100ms polling loop that: drains the SIGCHLD self-pipe and reaps dead children, accepts one control connection from the Unix domain socket, respawns services whose restart timer has elapsed, and starts services whose dependencies have been satisfied.

CGM2PLAN Binary Format

The CGM2PLAN format is designed for zero-allocation, zero-parsing read access. The executor casts the `mmap` base pointer directly to a `struct plan_header*` and then indexes into the step record array by offset arithmetic. The string table holds all command strings, working directory paths, and environment variable strings as NUL-terminated entries; step

records hold offsets into this table rather than inline strings, keeping the step records at a fixed 128 bytes.

Field	Offset	Size	Description
magic	0	8 bytes	b"CGM2PLAN" — identifies file type
version	8	4 bytes	Format version (currently 1)
step_count	12	4 bytes	Number of step records following header
strtab_offset	16	8 bytes	Byte offset of string table from file start
strtab_len	24	8 bytes	Length of string table in bytes
variant	32	4 bytes	0=Binary, 1=Native
flags	36	4 bytes	Reserved for future use
reserved	40	24 bytes	Pad to 64 bytes
step_records[N]	64	128 bytes × N	Array of typed step records
string_table	var	variable	NUL-terminated command/path strings

6.2 Data Flow Diagram

6.2.1 DFD Level 0 — Context Diagram

At the highest level of abstraction, the system has three external entities: the Package Author (provides TOML metadata files), the Build System Operator (invokes cogman-planner and cogman-executor on the host), and the Runtime Operator (manages services via cogman-ctl on the target). The central process is the Cogman Build and Runtime System, which accepts package metadata as input and produces a running OS environment as output.

[Figure 6.3: DFD Level 0 — Context Diagram]

6.2.2 DFD Level 1 — Build Subsystem

Expanding the build subsystem, the Level 1 DFD reveals five processes: (1) Load and Validate Metadata — reads TOML and validates against the schema; (2) Resolve Dependency Graph — performs topological sort on the package DAG; (3) Enforce Policy — checks filesystem write paths and network policy; (4) Emit Binary Plan — serializes step records and string table to CGM2PLAN; (5) Execute Plan — maps plan into memory,

dispatches each step, writes manifest. The data stores involved are the Package Metadata Store (/packages/), the Plan Cache (/tmp/cogman-cache/), and the Staging Root (\$PKGROOT).

[Figure 6.4: DFD Level 1 — Build Subsystem]

6.3 UML Diagrams

6.3.1 Use Case Diagram

The Build Subsystem use case diagram identifies two actors: the Package Author and the Build Operator. The Package Author performs: Write Package Metadata (TOML), Define Build Steps, Define Installer Steps, and Declare Dependencies. The Build Operator performs: Invoke cogman-planner, Invoke cogman-executor, Inspect Plan Cache, and Debug Failed Step. The Runtime Supervisor use case diagram identifies the Runtime Operator and the System (automated). The Runtime Operator performs: List Services, Start Service, Stop Service, and Restart Service via cogman-ctl. The System actor automatically performs: Start Services on Boot, Restart Failed Services, Reap Orphan Processes, and Accept Control Connections.

[Figure 6.5: Use Case Diagram — Build Subsystem]

[Figure 6.6: Use Case Diagram — Runtime Supervisor]

6.3.2 Class Diagram

The planner class diagram shows the following key classes and relationships: Cli (entry point) composes Command (enum: Build, Install, Deploy). Command::Install instantiates PackageMetadata via MetadataLoader. PackageMetadata contains Identity, Builder, Installer, Policy, and Depends. RecursiveLoader uses PackageMetadata to populate a DependencyGraph. DependencyGraph is consumed by TopologicalSort, which emits a

build order `Vec<String>`. `PlanEmitter` takes the ordered metadata list and emits a CGM2PLAN binary.

The supervisor class diagram shows: `Supervisor` (`g_sup`, global singleton) contains a fixed-size array of `Service` structs (max 64). `Service` has fields: `name`, `command`, `type` (`SvcType` enum), `restart` (`SvcRestart` enum), `state` (`SvcState` enum), `pid`, `started_at`, `restart_at`, `exit_code`, `restart_count`, `deps` array, `env` array. `CtlServer` wraps the Unix domain socket file descriptor and exposes `accept()` and `dispatch()` methods. The five command handlers (`list`, `status`, `start`, `stop`, `restart`) are associated with `CtlServer`.

[Figure 6.7: Class Diagram — Planner]

[Figure 6.8: Class Diagram — Supervisor]

6.3.3 Sequence Diagram

The Package Build sequence diagram shows the following message sequence: Operator invokes `cogman-planner` with metadata path and output path. `cogman-planner` calls `MetadataLoader::load()`, which reads the TOML file. `MetadataLoader` returns `PackageMetadata` to `Planner`. `Planner` calls `Validator::validate()`, receiving `Ok` or `Err(ValidationError)`. `Planner` calls `RecursiveLoader::inject_root()`, building the dependency graph. `Planner` calls `TopologicalSort::resolve_order()`, receiving ordered build list. `Planner` calls `PolicyEnforcer::check()` for each package in the list. `Planner` calls `PlanEmitter::emit()` with steps list, writing CGM2PLAN to disk. Operator invokes `cogman-executor` with plan path. `Executor` calls `plan_open()` and `plan_validate()`. `Executor` iterates over step records, calling `execute_step()` for each. `execute_step()` dispatches to `exec_command()`, `mkdir_p()`, `copy_recursive()`, or `verify_step()`. `Executor` returns exit code 0 on success or 1 on first step failure.

[Figure 6.9: Sequence Diagram — Package Build Flow]

[Figure 6.10: Sequence Diagram — Service Start Flow]

6.3.4 Component Diagram

The component diagram shows the physical deployment of Cogman components in the rootfs. The Host Build Environment contains cogman-planner and cogman-executor as executable components connected by the CGM2PLAN file artifact. The Target Rootfs contains four executable components: cogman-supervisor (connected to the Linux kernel via the PID 1 interface), cogman-exec (connected to cogman-supervisor via the service spawn interface), cogman-planner (connected to cogman-exec via the plan file interface), and cogman-ctl (connected to cogman-supervisor via the Unix domain socket). The `/etc/cogman/services/` directory is a data component consumed by cogman-supervisor. The `/etc/cogman/plans/` directory is a data component consumed by cogman-planner.

[Figure 6.11: Component Diagram]

CHAPTER 7

PROJECT DESCRIPTION

7.1 Module Description

7.1.1 cogman-planner Module

cogman-planner is the build-time planning component of Rogue Linux. Implemented in approximately 1,200 lines of Rust across nine source modules (cli, metadata, graph, variants, plan, policy, tmp, butler, error), it converts TOML package metadata into a binary execution plan. The planner is invoked from the command line or from within a service file (as in `exec-probe.service`) and exits immediately after writing the plan.

The planning pipeline consists of six stages. Stage 1 (Load) uses the `serde` and `toml` crates to deserialize the TOML file into a `PackageMetadata` struct. Any missing required field or type mismatch produces an error at this stage. Stage 2 (Validate) performs semantic checks beyond `serde`'s type system: `name/version/category` must be non-empty, `source.file` must be non-empty, `build.steps` must be non-empty, `installer.steps` must be non-empty, and filesystem write paths must be absolute.

Stage 3 (Resolve) builds a directed acyclic graph of package dependencies. Each dependency declared in `[identity.depends]` is loaded recursively from the `packages/` directory tree. The `RecursiveLoader` detects cycles by tracking visited nodes. Stage 4 (Sort) performs a topological sort using Kahn's algorithm, producing an ordered list of package names in which each package appears after all its dependencies. Stage 5 (Policy) enforces security policy: the declared filesystem write paths must be prefixes of the `rootfs` target; packages that declare `network outbound = true` are flagged if the build steps do not require network access.

Stage 6 (Emit) iterates over the ordered package list, calls the appropriate variant planner (`binary::plan()` or `native::plan()`), collects all `PlanStep` structs, and serializes them to the CGM2PLAN binary format. A content-addressed cache keyed by FNV-1a hash of the metadata is consulted before emission; if a matching cached plan exists and the metadata has not changed, the cached plan is written to the output path without re-planning.

The butler module provides all user-facing output using styled terminal strings (`butler::greet()`, `butler::info()`, `butler::check()`, `butler::smooth()`, `butler::success()`, `butler::warn()`, `butler::fatal()`). This separation ensures that diagnostic messages can be easily redirected or reformatted without changing the core planning logic.

[Figure 7.1: cogman-planner Dependency Graph Resolution]

[Figure 7.2: Dependency Topological Sort Example]

7.1.2 cogman-executor Module

`cogman-executor` is the build-time execution component, implemented in approximately 1,144 lines of C11 across seven source files (`main.c`, `plan/plan.c`, `log/log.c`, `exec/proc.c`, `fs/fs.c`, `verify/verify.c`, and the shared messenger/). It takes a single command-line argument (the path to a CGM2PLAN file) and executes the plan step by step.

The startup sequence opens the plan file, maps it into memory using `mmap()` with `PROT_READ` protection, validates the magic bytes (`b"CGM2PLAN"`), version number, and step count, and then enters the step dispatch loop. Each `step_record` contains an op code, a fail_policy (`FAIL_ABORT` or `FAIL_WARN`), and offsets into the string table for the command, working directory, and environment strings.

The five operation handlers are: `exec_command()` (forks a child, execs `/bin/sh -c cmd`, waits for completion, returns the exit code), `mkdir_p()` (creates a directory and all parents using a portable recursive implementation), `copy_recursive()` (copies a directory tree using a

readdir/openat/sendfile loop with path traversal validation on every destination path), verify_step() (checks that a path exists using access(F_OK), optionally verifying a SHA-256 checksum), and rm_rf() (removes a directory tree using nftw() with the FTW_DEPTH | FTW_PHYS flags).

The path traversal guard in copy_recursive() iterates over each path component of the destination path, checking for bare '..' entries. Any destination path that contains '..' is rejected immediately with exit code 2, regardless of whether the resolved path would actually escape the staging root. This conservative check prevents symlink-based traversal attacks in addition to direct '..' traversal.

[Figure 7.3: cogman-executor Step Execution Loop]

[Figure 7.4: Path Traversal Guard Logic]

7.1.3 cogman-supervisor Module

cogman-supervisor is the runtime supervisor and PID-1 init daemon, implemented in approximately 1,561 lines of C11 across four source files (main.c, service.c, ctl.c, ctl_main.c). When the Linux kernel finishes its initialization sequence, it forks and execs /sbin/init. On the Rogue Linux rootfs, /sbin/init is a symbolic link to cogman-supervisor, making it the first user-space process.

On startup, cogman-supervisor mounts the essential virtual filesystems (/proc as procfs, /sys as sysfs, /dev as devtmpfs, /run as tmpfs) when running as PID 1, loads all *.service files from /etc/cogman/services/, sets up signal handlers using sigaction() with the self-pipe SIGCHLD pattern, initializes the Unix domain socket at /run/cogman-supervisor.sock, and calls sup_start_ready() to start all services with no declared dependencies.

The SIGCHLD self-pipe pattern is central to the supervisor's correctness. A traditional approach of calling `waitpid()` from within the SIGCHLD handler is problematic because the handler interrupts the main thread at an arbitrary point, and many C library functions are not async-signal-safe. The self-pipe pattern solves this by having the handler write a single byte to a non-blocking pipe. The main loop uses `select()` to poll the read end of the pipe; when data is available, it drains the pipe and calls `waitpid(WNOHANG)` in a loop until no more dead children remain. All state modifications happen in the main loop, not in the signal handler.

The service state machine has six states: STOPPED (initial), STARTING (fork called), RUNNING (exec confirmed, PID is live), RESTARTING (waiting for restart_at deadline), FAILED (exited, restart policy is never or on-failure with exit code 0), and DONE (oneshot completed successfully). State transitions are driven by `svc_spawn()` (STOPPED → STARTING → RUNNING), SIGCHLD (RUNNING → RESTARTING or FAILED or DONE), `sup_tick()` (RESTARTING → RUNNING after delay), and `svc_stop()` (RUNNING → STOPPED).

[Figure 7.5: cogman-supervisor Signal Handling (Self-Pipe)]

[Figure 7.6: Service Lifecycle State Machine]

7.1.4 cogman-ctl Module

`cogman-ctl` is a thin client binary that communicates with a running `cogman-supervisor` over the Unix domain socket at `/run/cogman-supervisor.sock`. It is implemented in approximately 142 lines of C11 in `ctl_main.c`. The client accepts a command verb and an optional service name as command-line arguments, connects to the socket, sends the command as a newline-terminated string, reads the response until the connection is closed, and exits with code 0 if the response ends in "OK" or code 1 if it ends in "ERR".

The supervisor-side `ctl.c` implements `ctl_init()` (creates and binds the socket, sets it non-blocking, `chmod 0666`) and `ctl_accept()` (accepts one connection per main loop iteration, reads a command line, dispatches to a command handler, closes the connection). The `SO_RCVTIMEO` socket option is set to 2 seconds on accepted connections to prevent a misbehaving client from blocking the supervisor.

Five command handlers are implemented: `cmd_list()` prints all services with their state, PID, and restart count, then writes "OK\n". `cmd_status()` prints detailed information for one service. `cmd_start()` starts a stopped service if its dependencies are satisfied. `cmd_stop()` sends `SIGTERM` to the service's process group and polls for up to 3 seconds, then sends `SIGKILL`. `cmd_restart()` calls `cmd_stop()` followed by `cmd_start()`.

[Figure 7.7: cogman-ctl Unix Socket Protocol]

7.1.5 Messenger IPC Module

The messenger module provides structured inter-process communication between cogman components. It is implemented in approximately 277 lines of C11 across `messenger.c` and `client.c`. The protocol uses a fixed 16-byte message header followed by a variable-length payload.

The header structure contains: a 4-byte magic number (0x434F4731, ASCII 'COG1'), a 2-byte protocol version (currently 1), a 2-byte message type (`MSG_HEARTBEAT=0`, `MSG_HUD_ALERT=1`, `MSG_POLICY_REQ=2`, `MSG_DATA_XFER=3`, `MSG_LOG_INFO=4`), a 4-byte payload length, and a 4-byte source PID. The broker listens on a Unix domain socket at `/tmp/cogman.sock`. `messenger_broker_init()` creates this socket; `messenger_listen_and_process()` accepts connections and dispatches messages by type.

[Figure 7.8: Messenger IPC Protocol (TLV Format)]

7.1.6 Rootfs Bootstrap Module

The minimal Rogue Linux rootfs is a 6.3 MB directory tree that can boot under QEMU as an initramfs or as an ext4 image. The directory layout follows the Filesystem Hierarchy Standard (FHS) subset required by the service verification sequence: `/sbin/init` (symlink to `cogman-supervisor`), `/bin/sh` (symlink to `busybox`), `/bin/` (`busybox` applets), `/usr/bin/` (`cogman` binaries), `/lib/` (`glibc` + loader), `/etc/cogman/services/` (four service files), `/etc/cogman/plans/` (`smoke.toml`), `/dev/`, `/proc/`, `/sys/`, `/tmp/`, `/run/`, `/root/`.

The four verification services installed in `/etc/cogman/services/` exercise the complete service lifecycle: `hello.service` (oneshot, no deps) verifies the `fork/exec/SIGCHLD` reaping path; `heartbeat.service` (process, `restart=always`) verifies long-running process tracking and automatic restart; `ctl-probe.service` (oneshot, `depends=heartbeat`) verifies dependency gating and Unix socket IPC by calling `cogman-ctl list`; `exec-probe.service` (oneshot, `depends=ctl-probe`) verifies the complete planner-to-executor pipeline by generating and executing a smoke test plan.

[Figure 7.9: Rootfs Directory Layout]

CHAPTER 8

SYSTEM TESTING

8.1 Software Testing

System testing for Rogue Linux / Cogman is organized into six categories, reflecting the two-phase (build-time and runtime) architecture. The test suite is managed by `tests/run_all.py`, which discovers and executes all test modules, collects results, and produces a summary report. Tests are organized into subdirectories: `tests/metadata/`, `tests/graph/`, `tests/plan/`, `tests/executor/`, `tests/security/`, and `tests/perf/`.

8.1.1 Unit Testing

Unit tests verify individual functions and modules in isolation. The planner's metadata module is tested with over 400 test cases covering valid and invalid TOML inputs, missing required fields, type mismatches, and Unicode edge cases. The graph module is tested with 50+ cases including linear chains, diamond dependencies, and cycle detection. The plan module tests verify that the serialized binary layout matches the expected byte offsets for headers, step records, and string table entries.

The executor's C modules are tested using a dedicated test harness that invokes each operation handler with controlled inputs and verifies the return code, output files, and error messages. The path traversal guard is tested with 20+ cases including direct `'../'`, URL-encoded variants, and symlink chains.

8.1.2 Integration Testing

Integration tests verify the interaction between `cogman-planner` and `cogman-executor`. A reference package metadata file (`smoke.toml`) is used to generate a plan, which is then executed against a temporary staging directory. The test verifies that the expected output files

exist after execution, that the manifest records the correct file list, and that the executor exits with code 0.

A second integration test exercises the `cogman-supervisor` / `cogman-ctl` interaction. The supervisor is started in a subprocess with a temporary services directory containing one test service. After a brief delay, `cogman-ctl list` is invoked and the output is checked for the service name and a state of 'running' or 'done'. The supervisor is then sent `SIGTERM` and the test verifies that it exits cleanly.

8.1.3 System Testing

System testing runs the complete four-stage verification sequence on the minimal rootfs under QEMU. The QEMU command line is: `qemu-system-x86_64 -kernel /boot/vmlinuz -drive file=rootfs.img,format=raw -append 'console=ttyS0 root=/dev/sda rw init=/sbin/init quiet' -nographic -m 128M -serial mon:stdio`. The test captures the serial console output and searches for four expected strings: '[SERVICE:hello] cogman-supervisor is alive', '[SERVICE:heartbeat] tick 0', '[SERVICE:ctl-probe] control socket OK', and '[SERVICE:exec-probe] plan execution OK'.

[Figure 8.1: Service Boot Sequence on Minimal Rootfs]

8.1.4 Functional Testing

Functional testing verifies that each documented feature behaves as specified. Key functional test areas include: `cogman-planner` correctly rejects package files with missing required fields; `cogman-planner` detects circular dependencies and reports the cycle; the plan cache returns the cached plan when metadata is unchanged; `cogman-executor` correctly handles `FAIL_WARN` policy (continues on failure) versus `FAIL_ABORT` (stops on failure); `cogman-supervisor` starts services in dependency order; `cogman-supervisor` restarts services

with `restart=always` after a crash; `cogman-ctl` correctly returns exit code 1 when the supervisor responds with ERR.

8.1.5 Non-Functional Testing

Performance benchmarks were conducted comparing the Rogue Linux implementation against the Python reference implementation:

Metric	Python Reference	Rogue Linux	Improvement
Plan resolution time	~450 ms	~8 ms	56×
Peak memory (planner)	~85 MB	~4 MB	21×
Exec overhead per step	~45 ms	~0.9 ms	50×
Rootfs size (minimal)	N/A	6.3 MB	—
Boot time to first service	N/A	< 2 sec	—
Supervisor binary size	N/A	828 KB	Static, zero deps
Planner binary size	N/A	1.8 MB	Static Rust

Memory and CPU usage of the supervisor was profiled using `valgrind --tool=massif`. With four services running, the supervisor's resident set size is 1.2 MB. CPU usage during idle (main loop sleeping 100 ms per iteration) is below 0.1%.

8.1.6 User Acceptance Testing

User acceptance testing was conducted by three independent testers who were provided with the repository and asked to: (1) build the toolchain using `make all`, (2) write a new package TOML file for a trivial program, (3) generate and execute a plan for that package, (4) write a new service file and test it using `cogman-ctl`. All three testers completed all four tasks within 45 minutes. Feedback was collected via a structured questionnaire. The most common positive responses were: the TOML format is intuitive, the butler diagnostic messages are helpful, and `cogman-ctl`'s output is easy to parse. The most common suggestion for improvement was to add tab completion for `cogman-ctl` commands.

8.2 Test Cases

The following table presents representative test cases from each testing category:

TC ID	Test Name	Input / Action	Expected Output	Result
TC-001	Planner: valid package metadata	Pass valid smoke.toml	Exit 0, plan file written	Pass
TC-002	Planner: missing identity.name	TOML without [identity] name	Exit 1, ERR missing name	Pass
TC-003	Planner: circular dependency	A depends B, B depends A	Exit 1, cycle detected message	Pass
TC-004	Planner: cache hit	Plan same metadata twice	2nd run: 'served from cache'	Pass
TC-005	Planner: policy deny	Write path outside rootfs	Exit 1, policy denied message	Pass
TC-006	Executor: OP_EXEC success	Plan with echo command	Exit 0, output logged	Pass
TC-007	Executor: OP_MKDIR	Plan creates /tmp/test-dir	Directory exists after run	Pass
TC-008	Executor: OP_COPY path traversal	Copy src to dst/../../etc	Exit 2, traversal rejected	Pass
TC-009	Executor: OP_VERIFY existing file	Verify /etc/passwd	Exit 0	Pass
TC-010	Executor: OP_VERIFY missing file	Verify /nonexistent	Exit 1 if FAIL_ABORT	Pass
TC-011	Supervisor: service start (oneshot)	hello.service on boot	State DONE, exit_code 0	Pass
TC-012	Supervisor: restart=always	Kill heartbeat process	Restarted within 2+1 seconds	Pass
TC-013	Supervisor: dependency ordering	ctl-probe depends on heartbeat	ctl-probe starts after heartbeat	Pass
TC-014	Supervisor: cogman-ctl list	cogman-ctl list via socket	Lists all 4 services	Pass
TC-015	Supervisor: cogman-ctl stop	cogman-ctl stop heartbeat	heartbeat state: stopped	Pass
TC-016	Supervisor: cogman-ctl restart	cogman-ctl restart heartbeat	heartbeat state: running after cmd	Pass
TC-017	Supervisor: orphan reaping	Service forks grandchild then exits	Grandchild reaped by supervisor	Pass
TC-018	Supervisor: SIGTERM shutdown	Kill supervisor with SIGTERM	All services stopped, exit 0	Pass
TC-019	Rootfs: boot under QEMU	qemu-system-x86_64 with rootfs.img	All 4 verification messages appear	Pass
TC-020	Messenger: heartbeat IPC	Send MSG_HEARTBEAT from service	Broker ACK received	Pass

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The Rogue Linux / Cogman project has successfully demonstrated that a deterministic, safe, and high-performance system software stack can be constructed using modern Rust and C11 with no external runtime dependencies beyond the Linux kernel and the C standard library. The system achieves its primary design goals: a binary plan format that can be audited, cached, and executed with zero parsing overhead; a PID-1 supervisor that manages service dependencies and restarts with async-signal-safe signal handling; and a minimal rootfs that boots under QEMU and passes all verification tests in under two seconds.

The performance results validate the architectural choices. By replacing a Python-based plan generator with a Rust implementation and a shell-script executor with a C mmap executor, the system achieves a $56\times$ improvement in plan resolution time, a $21\times$ reduction in peak memory usage, and a $50\times$ reduction in per-step execution overhead. These improvements make Rogue Linux suitable for environments where build-time resources are constrained and where the build process itself must be fast enough to fit into a CI/CD pipeline.

The security properties of the system have been verified through targeted testing. The Rust type system enforces schema correctness at metadata load time. The path traversal guard in cogman-executor has been tested against 20+ attack vectors and correctly rejects all of them. The supervisor's SIGCHLD self-pipe pattern ensures that no unsafe function is called from a signal handler context.

The project also demonstrates the viability of a two-language approach to system software: Rust for the security-critical, complex planning logic where compile-time guarantees are most valuable, and C for the time-critical, kernel-interfacing execution and supervision logic where minimal runtime overhead is essential. This hybrid approach combines the best properties of both languages without requiring either to operate outside its strengths.

9.2 Future Enhancement

Multi-Architecture Support

The current implementation targets x86_64 exclusively. Extending cogman-planner's binary plan emitter and cogman-executor's process spawning to support ARM64 (aarch64) and RISC-V would make Rogue Linux viable for embedded single-board computers such as the Raspberry Pi 4 and RISC-V development boards. The primary change required is handling architecture-specific toolchain flags in the TOML build steps.

Package Signing and Verification

Currently, plan execution depends on SHA-256 checksums declared in the package TOML, which are not authenticated against a trusted key. Adding Ed25519 signature verification for both the TOML metadata and the CGM2PLAN output would provide an end-to-end integrity guarantee from package author to rootfs installation.

Persistent Package Database

The current rootfs has no persistent record of which packages have been installed. Adding a binary package database (similar to RPM's Berkeley DB or APK's index) would enable upgrade and removal operations. The cogman unified daemon already has a db module stub that can be extended for this purpose.

Landlock Filesystem Isolation

The cogman unified Rust daemon has a Landlock module that uses raw Linux syscalls 444–446 to restrict service filesystem access to the declared policy paths. Integrating Landlock enforcement into cogman-supervisor's `svc_spawn()` function would provide mandatory per-service filesystem isolation without requiring kernel namespaces or cgroups.

Health Probe Integration

The service file format supports a `[health]` section with `tcp`, `http`, and `exec` probe types, but the current cogman-supervisor implementation does not yet act on health check results. Implementing health probe execution and integrating it with the restart policy would allow services to be restarted based on behavioral correctness rather than merely process existence.

Interactive Shell Completion

User acceptance testing identified tab completion as the most requested feature for cogman-ctl. Implementing shell completion scripts for `bash`, `zsh`, and `fish` that query the live supervisor for current service names would significantly improve the operator experience.

AI Advisor Integration

An optional AI advisor using Qwen2.5-3B-Instruct (4-bit quantized via Ollama) is already prototyped in the cogman-advisor crate. Enabling this feature by default when an Ollama instance is available and extending it to diagnose not just build failures but also service crash analysis from the supervisor log would be a significant usability improvement.

CHAPTER 10

APPENDIX

10.1 Source Code

10.1.1 cogman-planner CLI Arguments (args.rs)

```
use clap::{Parser, Subcommand};
use std::path::PathBuf;

#[derive(Parser)]
#[command(name = "cogman", about = "Cogman build planner")]
pub struct Cli {
    #[command(subcommand)]
    pub command: Command,
}

#[derive(Subcommand)]
pub enum Command {
    Build {
        metadata: PathBuf,
        #[arg(long)] binary: bool,
        #[arg(short, long)] output: Option<PathBuf>,
        #[arg(long, default_value = "/mnt/rogue")] rootfs: String,
    },
    Install {
        metadata: PathBuf,
        #[arg(short, long)] output: Option<PathBuf>,
        #[arg(long, default_value = "/mnt/rogue")] rootfs: String,
    },
    Deploy {
        metadata: PathBuf,
        #[arg(short, long)] output: Option<PathBuf>,
        #[arg(long, default_value = "/mnt/rogue")] rootfs: String,
    },
}
```

10.1.2 cogman-supervisor Main Loop (main.c — key excerpt)

```
/* Main loop — 100 ms iterations */
struct timespec sleep_ts = { 0, 100000000L };
while (g_sup.running) {
    /* 1. Drain SIGCHLD pipe and reap dead children */
    fd_set rfd; FD_ZERO(&rfd);
    FD_SET(g_sigchld_pipe[0], &rfd);
    struct timeval tv = { 0, 0 };
    if (select(g_sigchld_pipe[0]+1, &rfd, NULL, NULL, &tv) > 0)
        drain_sigchld_pipe();

    /* 2. Accept and dispatch one control command */
    ctl_accept(g_sup.ctl_fd);

    /* 3. Respawn services whose restart timer has elapsed */
    sup_tick();

    /* 4. Start any service whose deps just became satisfied */
    sup_start_ready();
}
```



```

/* 5. Reap orphaned processes (PID 1 duty) */
int status; pid_t p;
while ((p = waitpid(-1, &status, WNOHANG)) > 0)
    sup_handle_dead(p, status);

nanosleep(&sleep_ts, NULL);
}

```

10.1.3 Service Lifecycle — svc_spawn() (service.c)

```

int svc_spawn(struct service *svc) {
    svc->state = SVC_STARTING;
    pid_t pid = fork();
    if (pid < 0) { svc->state = SVC_FAILED; return -1; }
    if (pid == 0) {
        /* Child: reset signals, redirect stdin, exec via sh -c */
        struct sigaction sa = {0};
        sa.sa_handler = SIG_DFL;
        for (int s = 1; s < 32; s++) sigaction(s, &sa, NULL);
        int null_fd = open("/dev/null", O_RDONLY);
        if (null_fd >= 0) { dup2(null_fd, STDIN_FILENO); close(null_fd); }
        char **envp = build_envp(svc);
        const char *sh_argv[] = { "/bin/sh", "-c", svc->command, NULL };
        execve("/bin/sh", (char *const *)sh_argv, envp);
        _exit(127);
    }
    /* Parent */
    svc->pid = pid; svc->state = SVC_RUNNING;
    svc->started_at = time(NULL);
    return 0;
}

```

10.1.4 Sample Service File — heartbeat.service

```

[service]
name      = heartbeat
command   = sh -c 'i=0; while true; do echo "[SERVICE:heartbeat] tick $i";
                                i=$((i+1)); sleep 5; done'
type      = process
restart   = always
restart_delay = 2

[env]
HEARTBEAT_TAG = rogue-linux
PATH = /usr/bin:/bin:/sbin:/usr/sbin

```

10.1.5 Sample Package Manifest — smoke.toml

```

[identity]
name      = "smoke"
version   = "0.0.1"
category  = "test"
summary   = "Minimal smoke-test plan"

[identity.source]
kind      = "none"

[identity.depends]
build     = []

[build]
system    = "custom"
steps     = []

```

```

[installer]
steps = [
    "echo '[SMOKE] step-1: OP_EXEC works'",
    "mkdir -p /tmp/cogman-smoke",
    "sh -c 'echo smoke-ok > /tmp/cogman-smoke/result'",
    "sh -c 'grep smoke-ok /tmp/cogman-smoke/result'",
]

[policy.filesystem]
read = ["/"]
write = ["/tmp"]

[policy.network]
outbound = false

```

10.1.6 CGM2PLAN Plan Header Structure (plan.h)

```

#define PLAN_MAGIC "CGM2PLAN"
#define PLAN_VERSION 1
#define VARIANT_BINARY 0
#define VARIANT_NATIVE 1

typedef enum { OP_EXEC=1, OP_MKDIR=2, OP_COPY=3,
               OP_VERIFY=4, OP_CLEANUP=5 } plan_op_t;
typedef enum { FAIL_ABORT=0, FAIL_WARN=1 } fail_policy_t;

struct __attribute__((packed)) plan_header {
    char magic[8]; /* "CGM2PLAN" */
    uint32_t version; /* 1 */
    uint32_t step_count;
    uint64_t strtabs_offset;
    uint64_t strtabs_len;
    uint32_t variant;
    uint32_t flags;
    uint8_t reserved[24];
};

struct __attribute__((packed)) step_record {
    uint8_t op;
    uint8_t fail_policy;
    uint8_t reserved[6];
    uint64_t cmd_offset;
    uint64_t wdir_offset;
    uint64_t env_offset;
    uint32_t env_len;
    uint32_t flags;
    uint8_t pad[80];
};

static_assert(sizeof(struct plan_header) == 64, "header must be 64 bytes");
static_assert(sizeof(struct step_record) == 128, "step must be 128 bytes");

```

REFERENCES

- [1] Poettering, L. and Sievers, K. (2010). systemd — Rethinking PID 1. Linux Symposium, Ottawa, Canada. Available at: <http://0pointer.de/public/systemd-paper.pdf>
- [2] Hennessy, J. L. and Patterson, D. A. (2019). Computer Organization and Design RISC-V Edition: The Hardware Software Interface. 5th ed. Morgan Kaufmann.
- [3] Klabnik, S. and Nichols, C. (2022). The Rust Programming Language, 2nd Edition. No Starch Press, San Francisco.
- [4] Kerrisk, M. (2010). The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.
- [5] Silberschatz, A., Galvin, P. B. and Gagne, G. (2018). Operating System Concepts, 10th Edition. John Wiley & Sons.
- [6] Linux Kernel Documentation Project (2024). kernel.org — init and early userspace. <https://www.kernel.org/doc/html/latest/admin-guide/init.html>
- [7] Coady, C. (2019). musl libc — A New Standard C Library. musl-libc.org. Available at: <https://musl.libc.org/doc/1.1.24/manual.html>
- [8] Buildroot Documentation (2024). Buildroot — making embedded Linux easy. buildroot.org. Available at: <https://buildroot.org/downloads/manual/manual.html>
- [9] ISO/IEC 9899:2011 (2011). Programming Languages — C (C11 Standard). International Organization for Standardization.
- [10] POSIX.1-2017 (2017). The Open Group Base Specifications Issue 7, 2018 edition. IEEE Std 1003.1-2017.
- [11] Serde Documentation (2024). serde — Serialization framework for Rust. <https://docs.rs/serde/latest/serde/>
- [12] Clap Documentation (2024). clap — A full featured command line argument parser for Rust. <https://docs.rs/clap/latest/clap/>
- [13] TOML Specification v1.0.0 (2021). Tom Preston-Werner. <https://toml.io/en/v1.0.0>
- [14] Torvalds, L. (2024). Linux Kernel Source: init/main.c. <https://github.com/torvalds/linux/blob/master/init/main.c>
- [15] Bernstein, D. J. (2004). Unix Application Environment. cr.yp.to. Available at: <https://cr.yp.to/unix.html>

CHAPTER 11

IMPLEMENTATION DETAILS

11.1 Build Environment Setup

Setting up the Rogue Linux build environment requires installing the Rust toolchain via `rustup` (<https://rustup.rs>) using the stable channel, GCC 11 or later, GNU Make 4.3, and Python 3.10 for the test runner. Once these prerequisites are installed, the entire toolchain is built with a single `make all` command from the repository root.

The top-level Makefile orchestrates the build in four phases. First, it builds the `cogman-planner` Rust binary using `cargo build --release` in the `cogman/src/planner` directory. Second, it builds `cogman-executor` using GCC with the `-O2 -std=c11` flags. Third, it builds `cogman-supervisor` and `cogman-ctl` using the same GCC flags. Fourth, it copies all four binaries to the `bin/` directory and optionally to `/usr/local/bin` with `make system-install`.

The Rust planner's `Cargo.toml` declares three external dependencies: `clap` 4.4 for argument parsing, `serde` 1.0 with the `derive` feature for automatic struct deserialization, and `toml` 0.8 for parsing TOML files into `serde`'s `Deserializer` trait. All three crates are pure Rust with no C dependencies, ensuring that the resulting binary is fully self-contained. The release build uses link-time optimization (`lto = true` in `Cargo.toml`) and `codegen-units = 1`, producing a single optimized binary.

The C executor and supervisor are linked statically using `-static-libgcc` to eliminate the runtime dependency on `libgcc_s.so`. In the minimal rootfs, they are linked against `glibc` 2.38 and are therefore not fully static, but the `glibc` shared objects (`libc.so.6`, `ld-linux-x86-64.so.2`) are included in the rootfs's `lib/` directory. An alternative `musl`-based build eliminates this dependency entirely, reducing the total rootfs size to approximately 4.1 MB.

11.2 Dependency Graph Implementation

The planner's dependency graph is implemented as an adjacency list in the `graph/resolve.rs` module. The `RecursiveLoader` struct maintains a `HashMap` mapping package names to `PackageMetadata` structs and a `HashMap` representing directed edges (dependency $\rightarrow$ dependents). When `inject_root()` is called with a package's metadata, it recursively loads all declared build dependencies from the `packages/` directory tree before adding the current package's node.

Dependency files are located by constructing the path `packages/<category>/<name>/<name>.toml` relative to the detected metadata root. The metadata root is inferred by walking up the directory tree from the input file path: the input file is assumed to be at `packages/<category>/<name>/<name>.toml`, so the root is four levels up. If the inference fails (for example, when running from a flat directory), the current working directory is used as the fallback root.

Cycle detection is performed using a depth-first search with a visited set. When the DFS encounters a node that is already in the current recursion stack (not just visited overall), it reports a cycle. The error message includes the cycle path in human-readable form so the package author can identify and resolve the circular dependency immediately.

Topological sort uses Kahn's algorithm, which builds a queue of nodes with in-degree zero (no remaining dependencies), processes them in order, and removes their outgoing edges to expose new zero-in-degree nodes. This produces a deterministic build order: given the same package graph, the topological sort always produces the same sequence, ensuring reproducible builds. The Kahn implementation in `topo.rs` returns an error if the queue empties before all nodes have been processed, which indicates a cycle that the DFS-based cycle detector should have caught earlier.

11.3 Binary Plan Format Implementation

The CGM2PLAN binary plan format is implemented in two places: the Rust emitter (`plan/emit.rs` and `plan/layout.rs`) and the C reader (`executor/plan/plan.c`). The Rust emitter writes the binary format to a `Vec<u8>` using explicit byte encoding with little-endian byte order for all integer fields. The `PlanStep` struct in `layout.rs` is the Rust representation; the `step_record` struct in `plan.h` is the C representation. They must be kept in sync manually — a `static_assert` in `plan.h` verifies the C struct size is exactly 128 bytes.

The string table is built incrementally as steps are added. A separate `HashMap` tracks `string → offset` mappings to deduplicate identical strings. For example, if 40 steps all have `workdir = '/tmp/build'`, the string `'/tmp/build'` appears only once in the string table, and all 40 step records point to the same offset. This deduplication typically reduces the string table size by 30–60% for packages with many similar steps.

The C reader maps the file using `mmap(NULL, file_size, PROT_READ, MAP_PRIVATE, fd, 0)`. The `PROT_READ` flag prevents accidental writes through the mapping. `MAP_PRIVATE` ensures that the mapping is copy-on-write; if the OS needs to reclaim the pages, the mapping will be re-populated from the file on next access. The `plan_str()` helper function takes a base pointer, a string table offset, and a per-string offset, and returns a `const char*` directly into the `mmap`'d region without any copying. Bounds checking ensures that the per-string offset does not exceed the declared string table length.

11.4 Service File Parser Implementation

The service file parser in `service.c` implements a simple INI-format reader without any external parsing library. Lines are read using `fgets()` into a 1024-byte buffer. Each line is trimmed of leading and trailing whitespace using the `trim()` helper, which modifies the string

in-place by advancing the start pointer and overwriting the trailing whitespace with a NUL byte.

Section headers are detected by checking whether the first non-whitespace character is '['. The section name is extracted by finding the matching ']' and NUL-terminating the string between them. The current section is tracked using an enum with values `SEC_NONE`, `SEC_SERVICE`, and `SEC_ENV`. Unknown section names are assigned `SEC_NONE` and silently ignored, making the parser forward-compatible with future service file versions that add new sections.

Key-value pairs are parsed by finding the first '=' character, splitting the line at that position, and trimming both sides. String values are stored using `strncpy` with explicit size limits to prevent buffer overflows. Integer values (`restart_delay`) are converted using `atoi()`, with a sanity check that the result is non-negative. Environment variable entries are stored as "KEY=VALUE" strings in the `env[]` array, using `snprintf` to combine the key and value with an '=' separator.

The environment merging logic in `build_envp()` combines the inherited process environment with the service-specific overrides. It first copies all inherited environment variables, skipping any whose key matches a key in the service's `env[]` array. It then appends the service-specific variables. This approach ensures that service-defined variables always take precedence over inherited ones without requiring the service author to re-declare variables they do not wish to override.

11.5 Control Socket Protocol Implementation

The control socket uses `AF_UNIX SOCK_STREAM` (a byte-stream connection-oriented Unix domain socket) rather than `SOCK_DGRAM`. The stream model simplifies the

protocol: the client sends a single command line, the server sends zero or more response lines followed by a terminal 'OK\n' or 'ERR <message>\n', and then closes the connection. The client reads until the connection is closed.

The supervisor's `ctl_accept()` function is called once per main loop iteration. It uses the non-blocking `accept()` call: if no client is waiting, `accept()` returns `EAGAIN` immediately and the function returns without delay. If a client is waiting, the socket is accepted, processed, and closed within the same main loop iteration. This single-connection-per-iteration model ensures that a slow or malicious client cannot block the supervisor's main loop for more than the 2-second `SO_RCVTIMEO` timeout.

The `write_all()` helper used throughout `ctl.c` handles partial writes by retrying the `write()` system call until all bytes have been sent or an error occurs. Partial writes can occur on Unix sockets when the kernel's socket buffer is full, which typically happens only under memory pressure. The `write_all()` function treats `EINTR` (interrupted by a signal) as a retry trigger, ensuring that the output is never silently truncated.

11.6 Rootfs Construction Process

The minimal Rogue Linux rootfs is constructed in three phases. Phase 1 creates the directory skeleton using `mkdir -p` for all required directories. Phase 2 populates the rootfs with pre-built binaries: the statically-linked `cogman-supervisor` is placed at `sbin/cogman-supervisor` and a symbolic link `sbin/init → cogman-supervisor` is created; `busybox` is placed at `bin/busybox` and symbolic links for all required applets (`sh`, `cat`, `echo`, `ls`, `mkdir`, `printf`, `ps`, `sleep`, `grep`, `test`) are created pointing to `bin/busybox`.

Phase 3 installs the dynamic libraries required by the non-statically-linked binaries. `cogman-planner` (the Rust binary) is statically linked and requires no libraries. `cogman-exec`

and cogman-ctl require glibc and the dynamic linker: libc.so.6 is copied from /lib/x86_64-linux-gnu/ on the build host, and ld-linux-x86-64.so.2 is placed at both lib/x86_64-linux-gnu/ and lib64/. libgcc_s.so.1 is included to support C++ exception handling in any future extensions. libresolv.so.2 is included for DNS resolution support.

The /etc directory contains the minimal files required for a working Unix environment: passwd (root user only), group (root group only), and hosts (localhost entry). The /etc/cogman/ tree contains the service definitions and plan files. The /tmp directory has permissions 01777 (sticky bit) to allow multiple processes to create files without interfering with each other. The /run directory is left empty; it will be mounted as a tmpfs by cogman-supervisor on boot.

The rootfs is packaged as an ext4 image using the finalize_rootfs.py script, which calculates the required image size based on the total file size plus a 20% overhead for filesystem metadata, formats the image using mkfs.ext4, and copies the rootfs tree into the image using e2cp. Alternatively, the rootfs directory can be used directly as an initramfs by passing it to QEMU's -initrd option, bypassing the need for an ext4 image entirely.

11.7 Performance Analysis

The $56\times$ improvement in plan resolution time (from ~ 450 ms to ~ 8 ms) arises from three factors. First, the Rust planner avoids Python's interpreter startup overhead (approximately 50 ms). Second, the TOML deserialization using serde is orders of magnitude faster than Python's toml library, which uses a recursive-descent parser written in pure Python. Third, the dependency graph algorithms use Rust's HashMap and Vec data structures with $O(1)$ average-case lookup and $O(V+E)$ graph traversal, compared to Python's dict-based implementation which has higher constant factors due to the interpreter's object model.

The $21\times$ reduction in peak memory (from ~ 85 MB to ~ 4 MB) is primarily attributable to Python's memory model: every Python object carries a reference count, a type pointer, and at least 24 bytes of overhead, making even simple string operations expensive. A Python string 'hello' requires approximately 57 bytes compared to 5 bytes in Rust's `str` type. For a package graph with hundreds of string fields across dozens of metadata structs, this difference accumulates to tens of megabytes.

The $50\times$ reduction in per-step execution overhead (from ~ 45 ms to ~ 0.9 ms) is dominated by the elimination of shell script parsing. The Python-based executor constructed a subprocess for each step using Python's `subprocess.run()`, which involves Python bytecode interpretation, object allocation, and multiple system calls to set up the subprocess environment. The C executor's `execute_step()` function calls `fork()` and `execve()` directly, taking approximately 0.5–1 ms on a modern x86_64 processor — essentially the minimum achievable overhead for process creation.

The plan cache provides an additional performance benefit on repeated builds. The FNV-1a hash is computed over the package name, version, and the content of the TOML file. On cache hit (unchanged metadata), plan emission takes approximately 0.3 ms — dominated by the file read and hash computation — compared to 8 ms for a full plan re-generation. In a continuous integration environment where most packages are unchanged between builds, the cache reduces total planning time by 90% or more.

11.8 Security Model

The Rogue Linux security model operates at two levels: policy enforcement during build planning and path safety enforcement during plan execution. At the planning level, the policy module in the planner checks that each package's declared filesystem write paths are

sub-paths of the declared rootfs target. A package that declares `write = ['/etc']` but is being installed into a rootfs at `/mnt/rogue` will be rejected unless `/etc` starts with `/mnt/rogue`. This prevents packages from accidentally (or maliciously) writing outside their staging area.

At the execution level, `path_has_traversal()` provides a first line of defense against path traversal attacks in `OP_COPY` operations. The function iterates over each slash-delimited path component, comparing it to the string `'..'`. This check is conservative: it rejects any path containing `'..'` regardless of whether the resolved absolute path would actually escape the staging root. This prevents symlink-based attacks where a component is a symlink to a directory that, combined with `'..'`, would escape the intended boundary.

The cogman-supervisor adds a runtime security layer by resetting all signal dispositions in child processes before calling `execve()`. This prevents a service from inheriting signal handlers set by the supervisor, ensuring that each service starts with a clean signal state. The supervisor also redirects `stdin` from `/dev/null` for all services, preventing services from accidentally reading from the terminal or blocking on a terminal read.

Future security enhancements include Landlock filesystem isolation (restricting each service to its declared filesystem policy paths using the Linux Landlock LSM), seccomp-based system call filtering (restricting each service to the minimum set of system calls required for its operation), and Linux namespaces (providing network and PID isolation for services that require it).

CHAPTER 12

EXTENDED TEST CASE DOCUMENTATION

12.1 Planner Test Cases — Schema Validation

The following test cases verify the schema validation behaviour of cogman-planner's metadata loading pipeline. Each test provides a specially crafted TOML input and verifies the planner's response. These tests ensure that malformed package definitions are rejected at planning time with a clear error message, preventing silent failures during build execution.

TC ID	Input Condition	Expected Behaviour	Pass/Fail
SV-001	Valid complete TOML	Exit 0, plan written	Pass
SV-002	Missing [identity] section	Exit 1, deserialization error	Pass
SV-003	identity.name = empty string	Exit 1, name must not be empty	Pass
SV-004	identity.version = empty string	Exit 1, version error	Pass
SV-005	identity.category contains space	Exit 1, invalid character	Pass
SV-006	build.steps = []	Exit 1, steps must not be empty	Pass
SV-007	installer.steps = []	Exit 1, installer steps error	Pass
SV-008	policy.filesystem.write = ['./etc']	Exit 1, non-absolute path	Pass
SV-009	identity.source.file = empty string	Exit 1, source file empty	Pass
SV-010	installer.verify.expected_files contains '..'	Exit 1, traversal in verify	Pass
SV-011	Circular dep A→B→A	Exit 1, cycle detected	Pass
SV-012	Dependency not found in packages/	Exit 1, missing dependency	Pass
SV-013	TOML syntax error (unclosed quote)	Exit 1, TOML parse error	Pass
SV-014	identity.depends.build contains empty string	Exit 1, empty build dep	Pass
SV-015	Valid TOML, policy.network.outbound = true	Exit 0, plan written (flagged)	Pass

12.2 Executor Test Cases — Step Operations

The following test cases verify the correct behaviour of each plan step operation type in cogman-executor. Tests are performed against a temporary staging directory using a specially constructed CGM2PLAN binary generated programmatically by the test harness.

TC ID	Operation	Test Condition	Expected Result	Pass/Fail
EX-001	OP_EXEC	echo 'test' command	stdout contains 'test'	Pass
EX-002	OP_EXEC	exit 1 command, FAIL_ABORT	Executor exits code 1	Pass

EX-003	OP_EXEC	exit 1 command, FAIL_WARN	Continues, logs warning	Pass
EX-004	OP_MKDIR	Create /tmp/t1/t2/t3	Directory exists	Pass
EX-005	OP_MKDIR	Already exists (idempotent)	Exit 0, no error	Pass
EX-006	OP_COPY	Copy file to valid dest	Dest file matches src	Pass
EX-007	OP_COPY	dest path contains '..'	Rejected, exit 2	Pass
EX-008	OP_COPY	Copy directory tree	All files copied	Pass
EX-009	OP_VERIFY	Existing file path	Exit 0	Pass
EX-010	OP_VERIFY	Non-existent path, ABORT	Exit 1	Pass
EX-011	OP_CLEANUP	Remove temp directory	Directory gone	Pass
EX-012	OP_CLEANUP	Non-existent dir, WARN	Continue, warn	Pass
EX-013	Header	Wrong magic bytes	Exit 1, bad magic	Pass
EX-014	Header	Wrong version number	Exit 1, version mismatch	Pass
EX-015	Header	step_count = 0	Exit 0, no steps	Pass

12.3 Supervisor Test Cases — Service Lifecycle

TC ID	Test Scenario	Setup	Expected Outcome	Pass/Fail
SL-001	Oneshot completes successfully	hello.service, type=oneshot	State transitions to DONE	Pass
SL-002	Oneshot fails, restart=never	Command exits 1, restart=never	State transitions to FAILED	Pass
SL-003	Process tracked by PID	Long-running sleep command	PID is positive, state=RUNNING	Pass
SL-004	restart=always after crash	Kill service process	Restart within delay+1s	Pass
SL-005	restart=on-failure, exit 0	Service exits cleanly	No restart	Pass
SL-006	restart=on-failure, exit 1	Service exits with error	Restart triggered	Pass
SL-007	Dependency gate blocks start	B depends A, A not started	B stays STOPPED	Pass
SL-008	Dependency gate opens	A completes (DONE)	B starts automatically	Pass
SL-009	Dependency chain A→B→C	All three services	Start order: A, B, C	Pass
SL-010	Explicit stop disables restart	cogman-ctl stop service	Does not restart	Pass
SL-011	Orphan reaping	Service forks grandchild	Grandchild reaped when orphaned	Pass
SL-012	SIGTERM initiates shutdown	kill -TERM PID_1	All services stopped cleanly	Pass
SL-013	Control socket unavailable	--no-ctl flag	Supervisor runs without socket	Pass
SL-014	Max 64 services loaded	64 .service files in dir	All 64 loaded successfully	Pass
SL-015	Invalid service file skipped	.service with no command field	Other services load normally	Pass

12.4 End-to-End Boot Test Cases

The following test cases verify the complete boot sequence on the minimal Rogue Linux rootfs under QEMU. Each test is conducted by booting the rootfs image with specific parameters and monitoring the serial console output for expected strings within a 30-second timeout window.

TC ID	Boot Condition	Expected Console Output	Pass/Fail
E2E-001	Normal boot, all services enabled	'[SERVICE:hello] cogman-supervisor is alive'	Pass
E2E-002	Normal boot	'[SERVICE:heartbeat] tick 0'	Pass
E2E-003	Normal boot	'[SERVICE:ctl-probe] control socket OK'	Pass
E2E-004	Normal boot	'[SERVICE:exec-probe] plan execution OK'	Pass
E2E-005	Boot with --no-ctl flag	ctl-probe fails, exec-probe never starts	Pass
E2E-006	Kill heartbeat (SIGKILL)	Supervisor restarts heartbeat in 3s	Pass
E2E-007	cogman-ctl list over network	All 4 services listed correctly	Pass
E2E-008	cogman-ctl stop heartbeat	heartbeat state: stopped	Pass
E2E-009	cogman-ctl start heartbeat	heartbeat restarts from stopped	Pass
E2E-010	Rootfs on QEMU with 64 MB RAM	All services start without OOM	Pass

CHAPTER 13

ADDITIONAL SOURCE CODE LISTINGS

13.1 Messenger Protocol Header (messenger.h)

```
#ifndef COGMAN_MESSENGER_H
#define COGMAN_MESSENGER_H

#include <stdint.h>

#define COGMAN_IPC_SOCKET "/tmp/cogman.sock"
#define IPC_MAGIC 0x434F4731 /* "COG1" */
#define IPC_ACK 0
#define IPC_NAK -1

typedef enum {
    MSG_HEARTBEAT = 0,
    MSG_HUD_ALERT = 1,
    MSG_POLICY_REQ = 2,
    MSG_DATA_XFER = 3,
    MSG_LOG_INFO = 4
} ipc_msg_type_t;

#pragma pack(push, 1)
struct ipc_header {
    uint32_t magic; /* IPC_MAGIC */
    uint16_t version; /* protocol version */
    uint16_t type; /* ipc_msg_type_t */
    uint32_t payload_len; /* bytes after hdr */
    uint32_t source_id; /* sender PID */
};
#pragma pack(pop)

int messenger_broker_init(void);
int messenger_listen_and_process(int server_fd);
int messenger_send(ipc_msg_type_t type, const void *payload, uint32_t len);
int messenger_hud_notify(const char *msg);

#endif /* COGMAN_MESSENGER_H */
```

13.2 Plan Validation Function (plan.c)

```
int plan_validate(const void *base, size_t size) {
    if (size < sizeof(struct plan_header)) {
        log_err("Plan file too small: %zu bytes", size);
        return -1;
    }
    const struct plan_header *hdr = (const struct plan_header *)base;
    if (memcmp(hdr->magic, PLAN_MAGIC, 8) != 0) {
        log_err("Bad magic: expected 'CGM2PLAN'");
        return -1;
    }
    if (hdr->version != PLAN_VERSION) {
        log_err("Version mismatch: got %u, expected %u",
            hdr->version, PLAN_VERSION);
        return -1;
    }
    size_t steps_end = sizeof(struct plan_header) +
        (size_t)hdr->step_count * sizeof(struct step_record);
```

```

    if (steps_end > size) {
        log_err("step_count %u overflows file size %zu", hdr->step_count, size);
        return -1;
    }
    if (hdr->strtab_offset + hdr->strtab_len > size) {
        log_err("String table extends beyond file end");
        return -1;
    }
    return 0;
}

```

13.3 Metadata Schema — Rust Struct Definitions (schema.rs)

```

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct PackageMetadata {
    pub identity: Identity,
    pub build: Builder,
    pub installer: Installer,
    #[serde(default)]
    pub policy: Policy,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Identity {
    pub name: String,
    pub version: String,
    pub category: String,
    pub summary: String,
    pub source: Source,
    #[serde(default)]
    pub depends: Depends,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Source {
    pub kind: SourceKind,
    pub file: String,
}

#[derive(Debug, Deserialize, Serialize, Clone, Copy, PartialEq)]
#[serde(rename_all = "lowercase")]
pub enum SourceKind { Tarball, Git }

#[derive(Debug, Deserialize, Serialize, Default, Clone)]
pub struct Depends {
    #[serde(default)] pub build: Vec<String>,
    #[serde(default)] pub runtime: Vec<String>,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Builder {
    pub system: BuildSystem,
    pub steps: Vec<String>,
}

#[derive(Debug, Deserialize, Serialize, Clone)]
pub struct Installer {
    pub steps: Vec<String>,
    pub verify: Option<Verify>,
}

#[derive(Debug, Deserialize, Serialize, Default, Clone)]
pub struct Policy {

```



```

    pub filesystem: Filesystem,
    pub network: Network,
}

```

13.4 Topological Sort — Kahn's Algorithm (topo.rs)

```

pub fn resolve_order(graph: &DependencyGraph) -> Result<ResolveResult, PlannerError> {
    let mut in_degree: HashMap<&str, usize> = HashMap::new();
    for node in graph.nodes() { in_degree.insert(node, 0); }
    for (_, deps) in graph.edges() {
        for dep in deps { *in_degree.entry(dep).or_insert(0) += 1; }
    }
    let mut queue: VecDeque<&str> = in_degree.iter()
        .filter(|(&_, &d)| d == 0)
        .map(|(n, _)| *n)
        .collect();
    let mut order = Vec::new();
    while let Some(node) = queue.pop_front() {
        order.push(node.to_string());
        if let Some(deps) = graph.depends(node) {
            for dep in deps {
                let d = in_degree.get_mut(dep).unwrap();
                *d -= 1;
                if *d == 0 { queue.push_back(dep); }
            }
        }
    }
    if order.len() != graph.node_count() {
        return Err(PlannerError::CyclicDependency(
            "Cycle detected in dependency graph".into()
        ));
    }
    Ok(ResolveResult { order })
}

```

13.5 Path Traversal Guard (executor/main.c)

```

static int path_has_traversal(const char *path) {
    const char *p = path;
    while (*p) {
        while (*p == '/') p++; /* skip leading/repeated slashes */
        if (p[0] == '.' && p[1] == '.' &&
            (p[2] == '/' || p[2] == '\0'))
            return 1; /* ".." component found */
        while (*p && *p != '/') p++; /* skip current component */
    }
    return 0;
}

/* Usage in execute_step() — OP_COPY handler: */
if (path_has_traversal(src_path) || path_has_traversal(dst_path)) {
    log_err("COPY rejected: path traversal detected "
        "(src='%s' dst='%s')", src_path, dst_path);
    free(buf);
    return -1; /* abort this step */
}
rc = copy_recursive(src_path, dst_path);

```

13.6 Service State Machine — sup_handle_dead() (main.c)

```
void sup_handle_dead(pid_t pid, int status) {
    struct service *svc = find_service_by_pid(pid);
    if (!svc) {
        fprintf(stderr, "sup: reaped orphan pid=%d\n", pid);
        return;
    }
    int exit_code = WIFEXITED(status) ? WEXITSTATUS(status)
        : WIFSIGNALED(status) ? -WTERMSIG(status) : 0;
    svc->exit_code = exit_code;
    svc->pid = -1;

    /* Oneshot: mark DONE or FAILED */
    if (svc->type == SVC_TYPE_ONESHOT) {
        svc->state = (exit_code == 0) ? SVC_DONE : SVC_FAILED;
        return;
    }
    /* Forking: parent exit is intentional */
    if (svc->type == SVC_TYPE_FORKING) { svc->state = SVC_DONE; return; }

    /* Explicitly stopped — honour that */
    if (svc->state == SVC_STOPPED) return;

    /* Apply restart policy */
    int should = 0;
    switch (svc->restart) {
        case SVC_RESTART_ALWAYS: should = 1; break;
        case SVC_RESTART_ON_FAILURE: should = (exit_code != 0); break;
        default: should = 0; break;
    }
    if (should) {
        svc->state = SVC_RESTARTING;
        svc->restart_at = time(NULL) + svc->restart_delay;
    } else {
        svc->state = (exit_code == 0) ? SVC_STOPPED : SVC_FAILED;
    }
}
```

13.7 Minimal-Linux Distribution Manifest (minimal-linux.toml)

```
[distro]
name = "rogue-linux-minimal"
version = "0.1.0"
arch = "x86_64"
summary = "Minimal Rogue Linux image — cogman verification target"

[[packages]]
name = "musl"; version = "1.2.5"; category = "base"; toml = "musl/musl.toml"

[[packages]]
name = "busybox"; version = "1.36.1"; category = "base"; toml = "busybox/busybox.toml"

[[packages]]
name = "linux-kernel"; version = "6.6.30"; category = "base"
toml = "linux-kernel/linux-kernel.toml"

[[packages]]
name = "cogman-exec"; version = "1.0.0"; category = "cogman"
toml = "cogman-exec/cogman-exec.toml"

[[packages]]
```

```
name = "cogman-planner"; version = "1.0.0"; category = "cogman"  
toml = "cogman-planner/cogman-planner.toml"
```

```
[[packages]]  
name = "cogman-supervisor"; version = "1.0.0"; category = "cogman"  
toml = "cogman-supervisor/cogman-supervisor.toml"
```

```
[rootfs]  
makedirs = ["/bin", "/sbin", "/usr/bin", "/lib", "/etc/cogman/services",  
            "/etc/cogman/plans", "/run", "/tmp", "/proc", "/sys", "/dev", "/root"]  
init = "/sbin/init"
```

```
[image]  
kernel = "/boot/vmlinuz"  
cmdline = "console=ttyS0 root=/dev/sda rw init=/sbin/init quiet"
```

CHAPTER 14: PROJECT METRICS AND ANALYSIS

14.1 Lines of Code Analysis

The Rogue Linux / Cogman codebase comprises approximately 4,082 lines of code across two programming languages. The following table presents a detailed breakdown by component and language. This analysis excludes auto-generated code, build artifacts, documentation files, and test data files. Only production source code in .rs, .c, and .h files is counted.

Component	Language	Source Files	Lines of Code	Key Functionality
cogman-planner	Rust	9 files	~1,200 LOC	Plan generation, dep graph, policy
cogman-executor	C11	7 files	~1,144 LOC	Plan execution, path safety, mmap
cogman-supervisor	C11	4 files	~1,184 LOC	PID-1 init, service lifecycle
cogman-ctl	C11	1 file	~142 LOC	Unix socket control client
messenger	C11	3 files	~277 LOC	TLV IPC protocol
service files	TOML	4 files	~135 LOC	Service definitions
Total	Mixed	28 files	~4,082 LOC	Complete system

The ratio of C to Rust code (approximately 2,747 C lines vs. 1,200 Rust lines) reflects the design philosophy: Rust is used only where its type-system guarantees add significant value (the planner's dependency resolution and schema validation), while C is used for all kernel-interface and performance-critical code.

The test suite adds approximately 1,800 additional lines of Python (test runner and test cases) and 400 lines of C (executor test harness). Test coverage is estimated at 78% for the planner, 72% for the executor, and 65% for the supervisor, measured by line coverage using gcov and llvm-cov.

14.2 Build Time Analysis

Build times were measured on a reference machine running Kali Linux 6.x on an Intel Core i7-11800H (8 cores, 4.6 GHz boost) with 16 GB DDR4 RAM and a Samsung 970 EVO NVMe SSD. The measurements represent wall-clock time for a clean build (no cached artifacts) averaged over five runs.

Component	Build Command	Clean Build Time	Incremental (1 file)
cogman-planner	cargo build --release	28.4 s	4.2 s
cogman-executor	make -C cogman/src/executor	1.1 s	0.3 s
cogman-supervisor	make -C cogman/src/supervisor	0.9 s	0.2 s
Full (make all)	make all	31.8 s	5.1 s

The Rust planner dominates build time because the Rust compiler performs exhaustive type checking, lifetime analysis, and link-time optimization. The C components build in under two seconds combined. Incremental builds after a single file change are significantly faster thanks to Cargo's dependency tracking and GCC's implicit dependency generation via -MD flags.

14.3 Binary Size Analysis

Binary sizes were measured on the compiled ELF executables before stripping and after stripping debug symbols. Stripping is performed automatically as part of the rootfs construction process.

Binary	Unstripped	Stripped	Linking	Dependencies
cogman-supervisor	1.2 MB	828 KB	Dynamic (glibc)	libc.so.6
cogman-executor	1.1 MB	782 KB	Dynamic (glibc)	libc.so.6
cogman-planner	6.2 MB	1.8 MB	Static (Rust)	None
cogman-ctl	1.1 MB	799 KB	Dynamic (glibc)	libc.so.6
busybox	1.8 MB	803 KB	Static	None
Total (rootfs)	11.4 MB	~5.0 MB	Mixed	~1.3 MB libs

The large unstripped size of cogman-planner (6.2 MB) is primarily due to the extensive DWARF debug information generated by rustc. After stripping, the binary compresses to 1.8 MB — reasonable for a Rust binary that links the clap, serde, and toml crates statically.

14.4 Memory Usage Profiling

Memory usage was profiled using Valgrind's Massif tool (heap profiler) and `/proc/<pid>/status` (RSS measurement). The supervisor was profiled while managing four services in the steady state (all services started, heartbeat running, ctl-probe and exec-probe completed).

Component	Heap Peak	Stack Peak	RSS (steady state)	Leak Check
cogman-supervisor	64 KB	32 KB	1.2 MB	No leaks
cogman-executor	128 KB	48 KB	2.4 MB	No leaks
cogman-planner	3.8 MB	256 KB	4.1 MB	No leaks
cogman-ctl	4 KB	16 KB	0.6 MB	No leaks

Valgrind's memcheck tool reported zero memory leaks in cogman-supervisor across 72 hours of continuous operation with 1,000 service restart cycles. The supervisor's fixed-size service table ($64 \times \text{sizeof}(\text{struct service}) \approx 64 \times 640 \text{ bytes} = 40 \text{ KB}$) means that memory consumption does not grow with the number of service restarts.

14.5 Comparison with Existing Systems

The following table compares Rogue Linux / Cogman against the closest existing systems in the embedded Linux build and init space. Metrics are based on published documentation and independent benchmarks where available.

Feature	Rogue Linux	Buildroot	systemd	Busybox init	runit
Build language	Rust + C11	Make + sh	C + C++	C	C
Metadata format	TOML	Kconfig+Make	INI units	shell script	run scripts
Schema validation	Compile-time	None	Runtime	None	None

Binary plan format	CGM2PLAN	None	None	None	None
Dep graph resolution	8 ms	N/A	Boot-time	N/A	None
Peak memory (plan)	4 MB	N/A (host)	N/A	N/A	N/A
PID-1 capable	Yes	No (uses init)	Yes	Yes	Yes
Unix socket ctl	Yes	No	Yes (dbus)	No	No
Restart policies	3	N/A	Many	1 (respawn)	1 (always)
Dep ordering	Yes	N/A	Yes	No	No
Init binary size	828 KB	N/A	1.5 MB	~100 KB	~30 KB
AI failure diagnosis	Optional	No	No	No	No